Concurrency

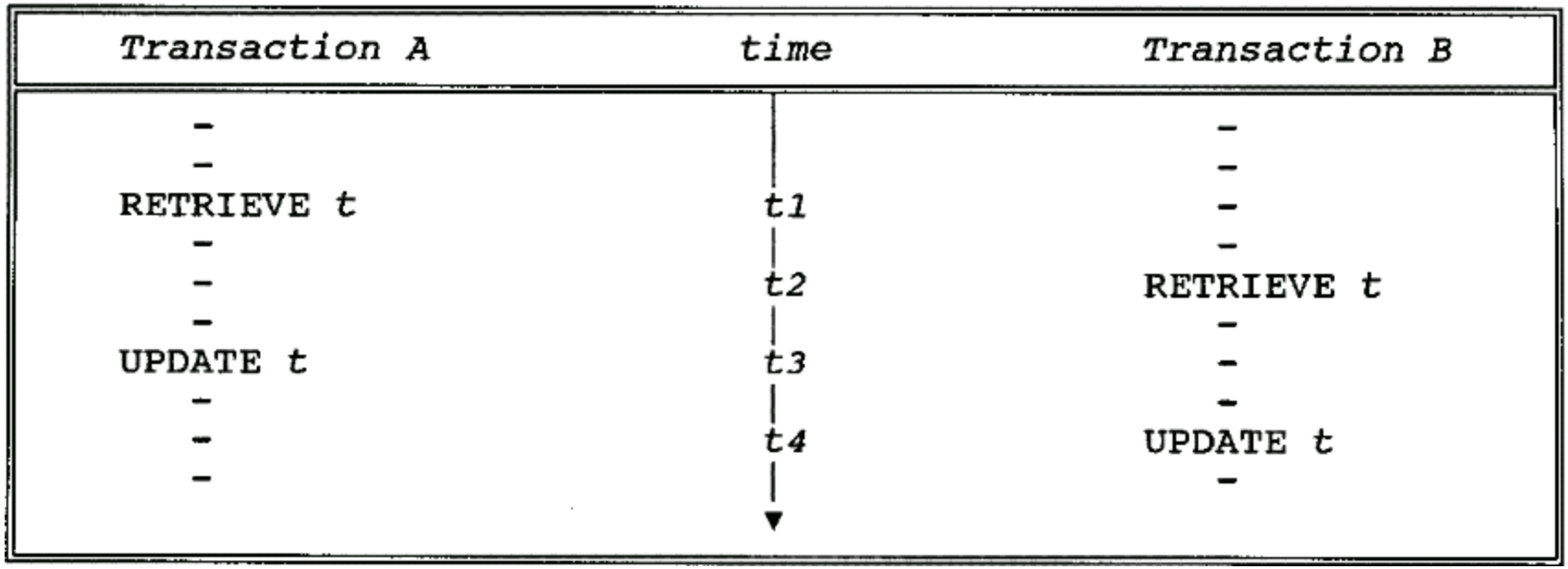
1. Introduction

- concurrency : DBMSs typically allow many transactions to access the same database at the same time.
- concurrency control mechanism : ensure that concurrent transactions do not interfere with each other.

2.Three Concurrency Problems

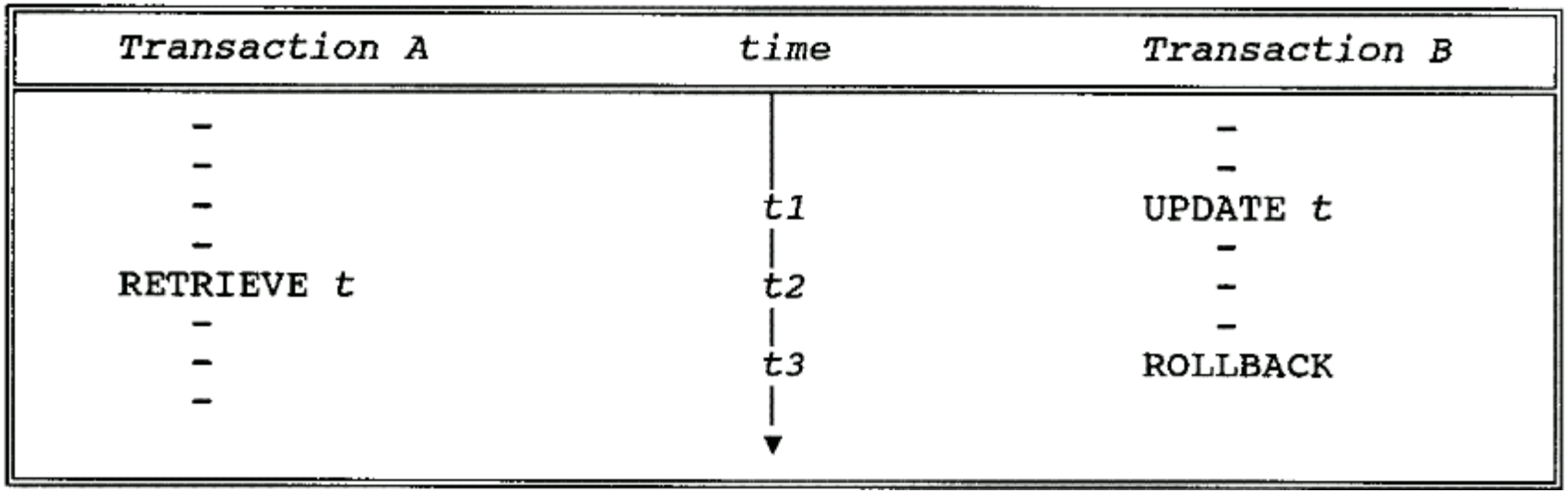
- The lost update problem
  (丢失修改)
- The uncommitted dependency problem
  (读“脏”数据问题)
- The inconsistent analysis problem
  (不可重复读问题)

The Lost Update Problem



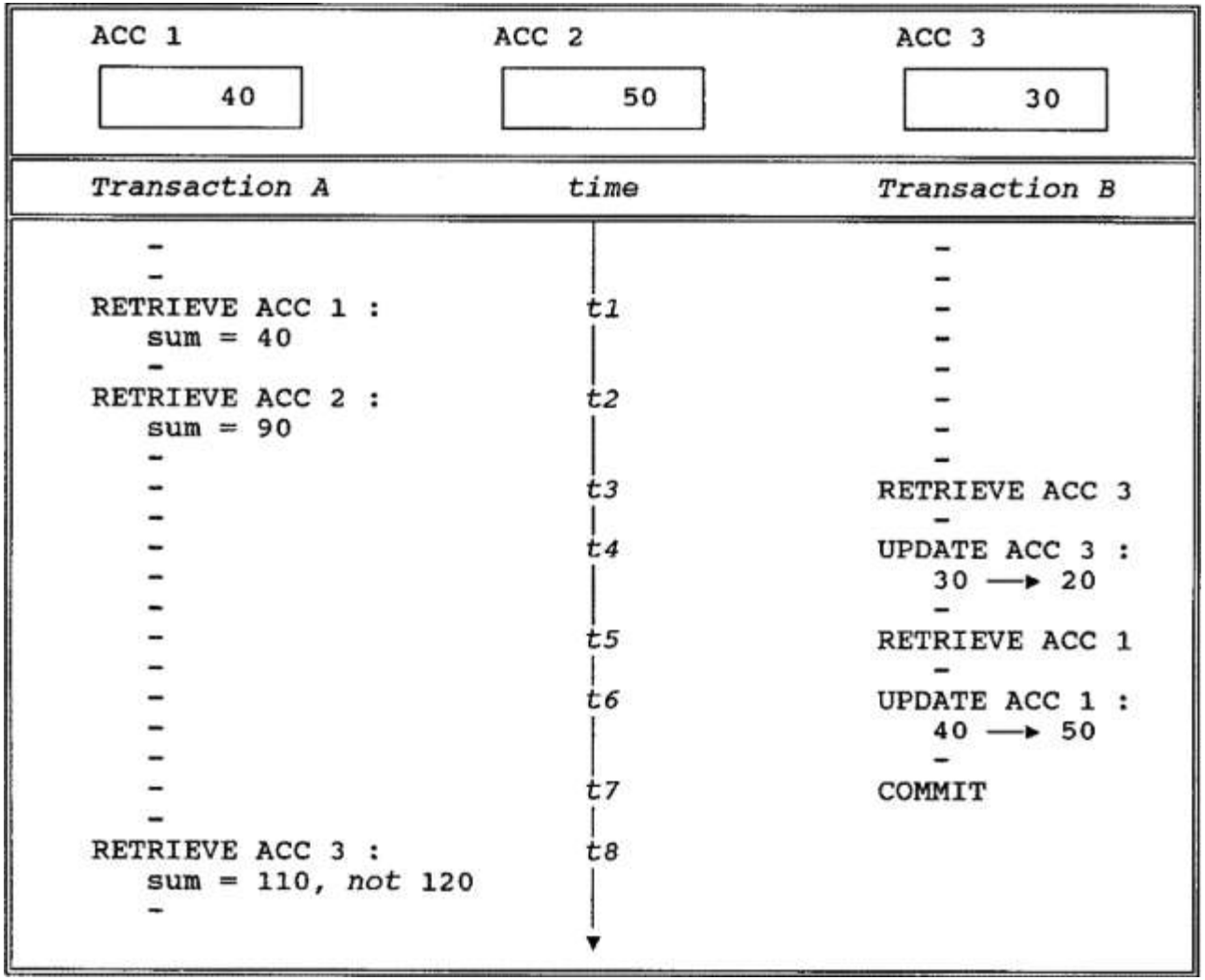
- Transaction A's update is lost at time t4.

The Uncommitted Dependency Problem



- The uncommitted dependency problem arises if one transaction is allowed to retrieve—or, worse, update—a tuple that has been updated by another transaction but not yet committed by that other transaction.

The Inconsistent Analysis Problem



3. Locking (锁)

- Two kinds of locks:
  - Exclusive locks (X locks) / write locks
  - Shared locks (S locks) / read locks
- If transaction A holds an exclusive (X) lock on tuple t, then a request from some distinct transaction B for a lock of either type on t will be denied.
- If transaction A holds a shared (S) lock on tuple t, then:
  - A request from some distinct transaction B for an X lock on t will be denied;
  - A request from some distinct transaction B for an S lock on t will be granted (that is, B will now also hold an S lock on t).

Lock type compatibility matrix
(锁相容矩阵)

	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

Data access protocol / locking protocol
(封锁协议)

- A transaction that wishes to retrieve a tuple must first acquire an S lock on that tuple.
- A transaction that wishes to update a tuple must first acquire an X lock on that tuple.
  - If it already holds an S lock on the tuple, then it must promote that S lock to X level.

Data access protocol / locking protocol…

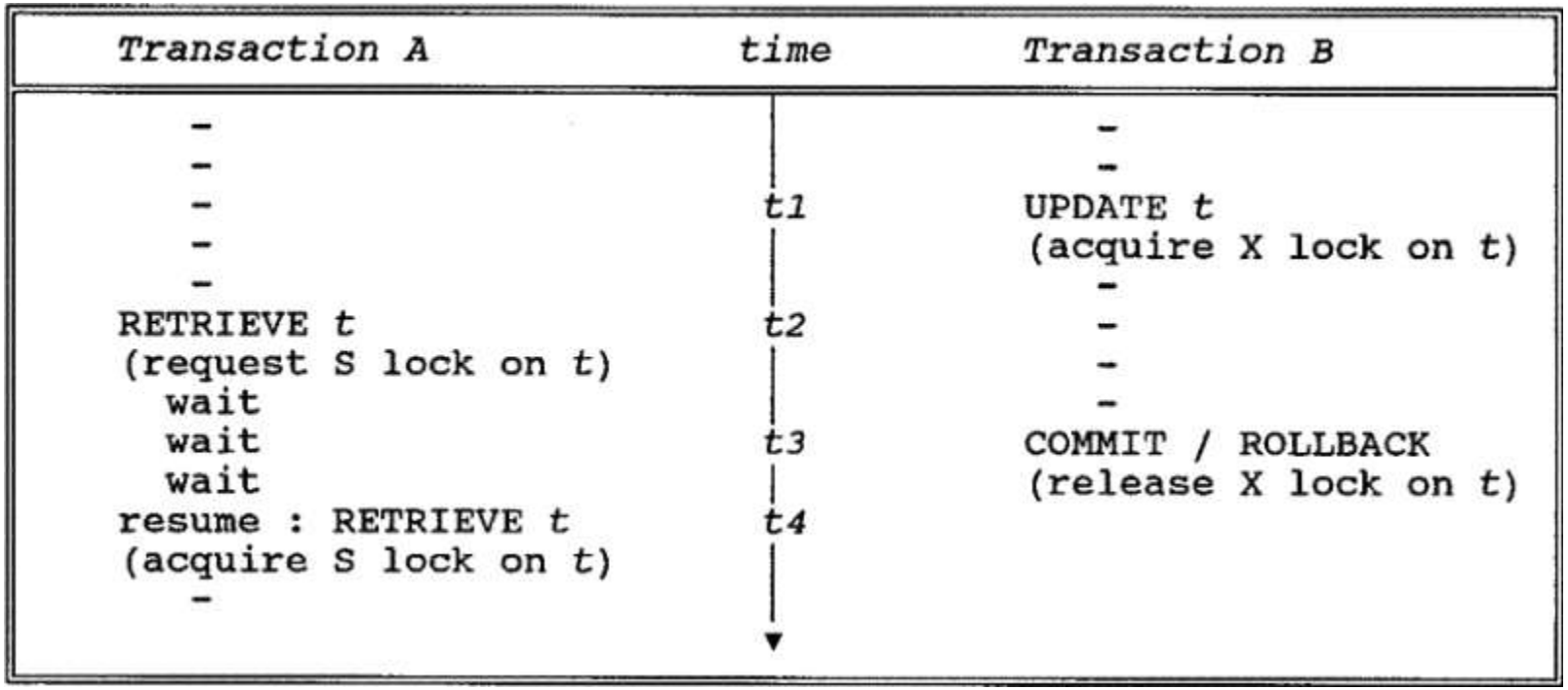
- If a lock request from transaction B is denied because it conflicts with a lock already held by transaction A, transaction B goes into a wait state. B will wait until A's lock is released.
  - The system must guarantee that B does not wait forever.
  - A simple way to provide such a guarantee is to service all lock requests on a "first-come, first-served" basis.
- X locks are held until end-of-transaction (COMMIT or ROLLBACK). S locks are normally held until that time.

4. The Three Concurrency Problems Revisited

Transaction A	time	Transaction B
-		-
-		-
RETRIEVE *t*	*t1*	-
(acquire S lock on *t*)		-
-		-
-	*t2*	RETRIEVE *t*
-		(acquire S lock on *t*)
-		-
UPDATE *t*	*t3*	-
(request X lock on *t*)		-
wait		-
wait	*t4*	UPDATE *t*
wait		(request X lock on *t*)
wait		wait
wait		wait
wait		wait

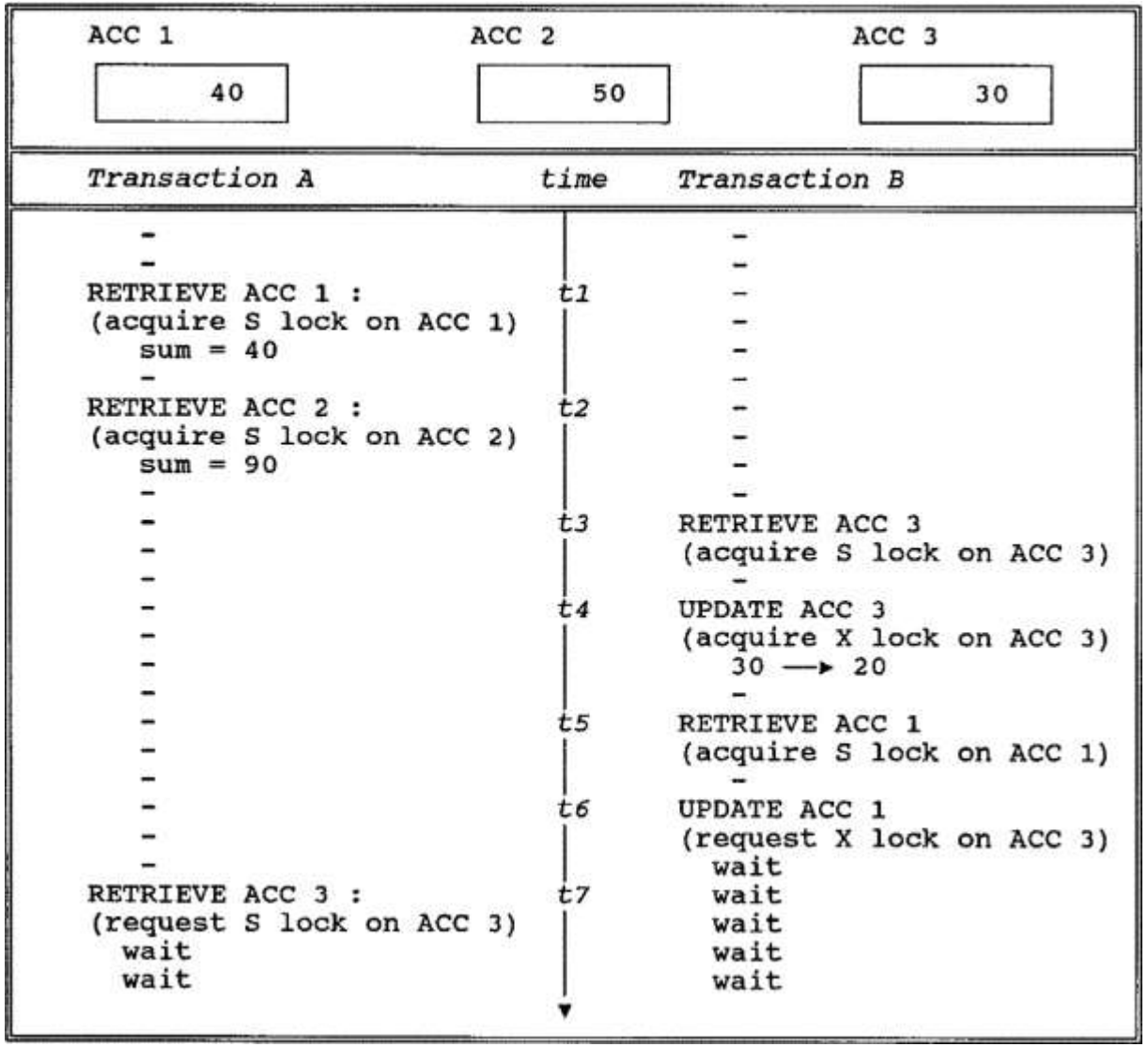
- Locking solves the lost update problem.
- Another problem: deadlock

The Uncommitted Dependency Problem

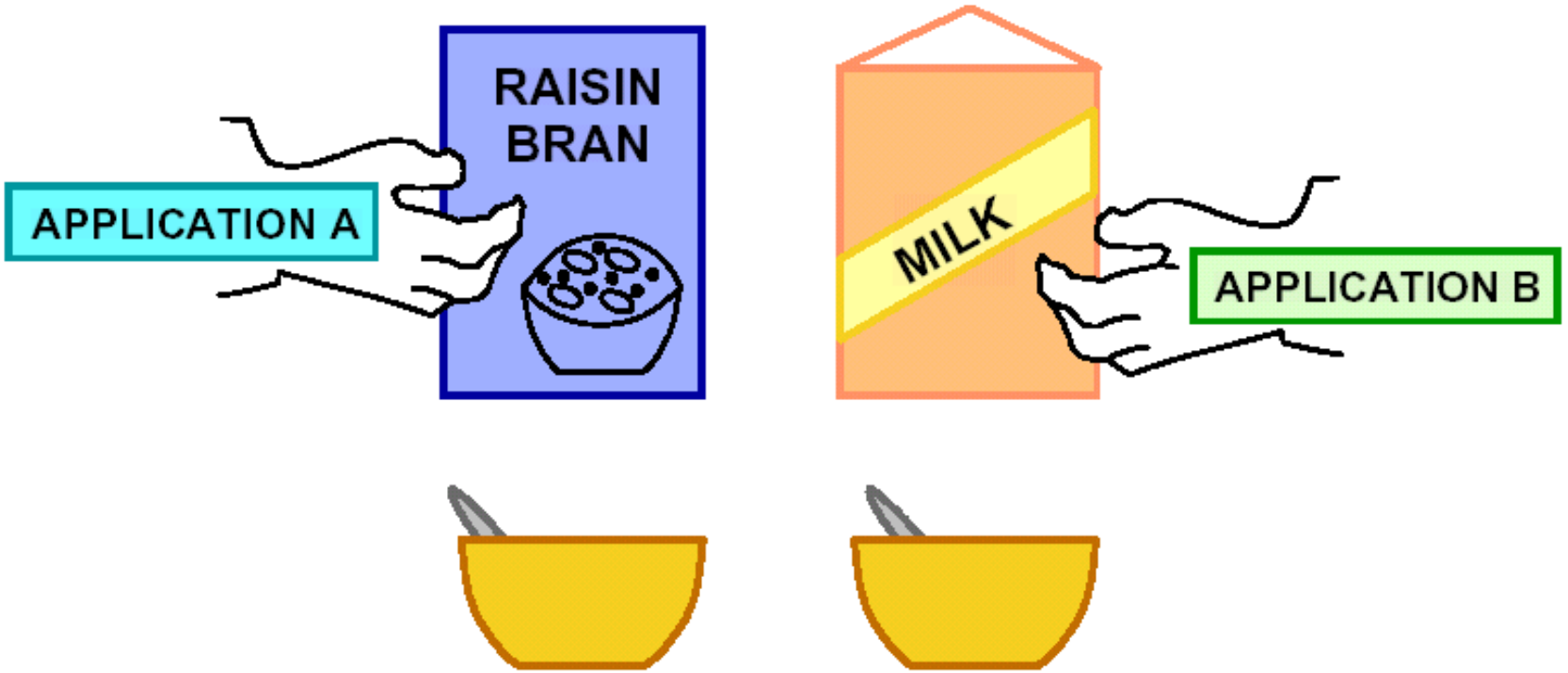


- Transaction A is prevented from seeing an uncommitted change at time t2.

The Inconsistent Analysis Problem



5. Deadlock



5. Deadlock …

- Deadlock is a situation in which two or more transactions are in a simultaneous wait state, each of them waiting for one of the others to release a lock before it can proceed.

Transaction A	*time*	*Transaction B*
-		-
-		-
LOCK *r1* EXCLUSIVE	*t1*	-
-		-
-	*t2*	LOCK *r2* EXCLUSIVE
-		-
LOCK *r2* EXCLUSIVE	*t3*	-
wait		-
wait	*t4*	LOCK *r1* EXCLUSIVE
wait		wait
wait		wait
	▼	

5. Deadlock …

- If a deadlock occurs, it is desirable that the system detect it and break it.
- Detecting the deadlock involves detecting a cycle in the Wait-For Graph.
- Breaking the deadlock involves choosing one of the deadlocked transactions as the victim and rolling it back, thereby releasing its locks and so allowing some other transaction to proceed.

死锁的检测（补充）

（1）超时法

如果一个事务的等待时间超过了规定的时限，就认为发生了死锁。

优点：实现简单。

缺点：一是有可能误判死锁；

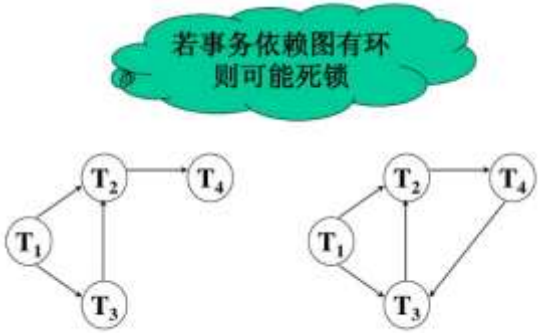
二是若时限设置的太长，可能出现死锁发生后不能及时发现的情况。

（2）有向等待图法

检测死锁

画一个表示事务等待关系的有向等待图G=(V, E)。其中V为结点集合，每个结点表示一个正在运行的事务E为有向边集合，每条有向边表示事务的等待关系：若T1正在等待给被T2锁住的数据项加锁，则在T1和T2之间划一条有向边，方向是T1指向T2。

事务的有向等待图动态地反映了所有事务的等待情况。



6. Serializability(可串行性)

- A given execution of a set of transactions is serializable(可串行化的), if it produces the same result as some serial execution of the same transactions, running them one at a time.
- A given execution of a set of transactions is considered to be correct if it is serializable.

6. Serializability(可串行性)…

- Individual transactions are assumed to be correct:
  - transform a correct state of the database into another correct state.
- Running the transactions one at a time in any serial order is correct:
  - because individual transactions are assumed to be independent of one another.
- An interleaved execution is correct if it is equivalent to some serial execution—i.e., if it is serializable.

6. Serializability(可串行性)…

- Terminology:
  - Given a set of transactions, any execution of those transactions, interleaved or otherwise, is called a schedule(调度).
  - Executing the transactions one at a time, with no interleaving, constitutes a serial schedule(串行调度);
  - A schedule that is not serial is an interleaved schedule并发调度 (or simply a nonserial schedule).
  - Two schedules are said to be equivalent if they are guaranteed to produce the same result, independent of the initial state of the database.

6. Serializability(可串行性)...

事务T1	事务T2	数据库中的值
read (A)		A=50
A:=A-2		
write (A)		A=48
read (B)		B=100
B:=B-1		
write (B)		B=99
	read (A)	A=48
	A:=A-2	
	write (A)	A=46
	read (B)	B=99
	B:=B-1	
	write (B)	B=98

串行调度1：先T1后T2

事务T1	事务T2	数据库中的值
	read (A)	A=50
	A:=A-2	
	write (A)	A=48
	read (B)	B=100
	B:=B-1	
	write (B)	B=99
read (A)		A=48
A:=A-2		
write (A)		A=46
read (B)		B=99
B:=B-1		
write (B)		B=98

串行调度2：先T2后T1

6. Serializability(可串行性)...

事务T1	事务T2	数据库中的值
read (A)		A=50
A:=A-2		
write (A)		A=48
	read (A)	
	A:=A-2	
	write (A)	A=46
read (B)		B=100
B:=B-1		
write (B)		B=99
	read (B)	
	B:=B-1	
	write (B)	B=98

并发调度1

事务T1	事务T2	数据库中的值
read (A)		A=50
A:=A-2		
write (A)		A=48
read (B)		B=100
	read (A)	A=48
	A:=A-2	
	write (A)	A=46
	read (B)	B=100
B:=B-1		
write (B)		B=99
	B:=B-1	
	write (B)	B=98

并发调度2

可串行化调度

现在有两个事务，分别包含下列操作：

- 事务T1：读B；A=B+1；写回A
- 事务T2：读A；B=A+1；写回B

现给出对这两个事务不同的调度策略

串行调度,正确的调度

T_1	T_2
Slock B	
Y=R(B)=2	
Unlock B	
Xlock A	
A=Y+1=3	
W(A)	
Unlock A	
	Slock A
	X=R(A)=3
	Unlock A
	Xlock B
	B=X+1=4
	W(B)
	Unlock B

串行调度(a)

- 假设A、B的初值均为2。
- 按$T_1 \rightarrow T_2$次序执行结果为A=3，B=4
- 串行调度策略,正确的调度

串行调度,正确的调度

T_1	T_2
	Slock A
	X=R(A)=2
	Unlock A
	Xlock B
	B=X+1=3
	W(B)
	Unlock B
Slock B	
Y=R(B)=3	
Unlock B	
Xlock A	
A=Y+1=4	
W(A)	
Unlock A	

串行调度(b)

- 假设A、B的初值均为2。
- $T_2 \rightarrow T_1$次序执行结果为B=3，A=4
- 串行调度策略,正确的调度

不可串行化调度，错误的调度

T_1	T_2
Slock B	
Y=R(B)=2	
	Slock A
	X=R(A)=2
Unlock B	
	Unlock A
Xlock A	
A=Y+1=3	
W(A)	
	Xlock B
	B=X+1=3
	W(B)
Unlock A	
	Unlock B

不可串行化的调度

- 执行结果与(a)、(b)的结果都不同
- 是错误的调度

可串行化调度，正确的调度

T_1	T_2
Slock B	
Y=R(B)=2	
Unlock B	
Xlock A	
	Slock A
A=Y+1=3	等待
W(A)	等待
Unlock A	等待
	X=R(A)=3
	Unlock A
	Xlock B
	B=X+1=4
	W(B)
	Unlock B

可串行化的调度

- 执行结果与串行调度(a)的执行结果相同
- 是正确的调度

7. Two-phase locking protocol

- If all transactions obey the "two-phase locking protocol," all possible interleaved schedules are serializable.
- Two-phase locking protocol:
  - Before operating on any object (e.g., a database tuple), a transaction must acquire a lock on that object;
  - After releasing a lock, a transaction must never go on to acquire any more locks.

“两段”锁的含义

– 事务分为两个阶段

- 第一阶段是获得加锁，也称为扩展阶段；
- 第二阶段是释放加锁，也称为收缩阶段。

7. Two-phase locking protocol…

- T1:

  Slock(A)、Slock(B)、Xlock(C)、Unlock(A)、Unlock(B)、Unlock(C)

  obey the two-phase locking protocol

- T2: Xlock(B)、Unlock(B)

  obey the two-phase locking protocol

- T3:

  Slock(A)、Slock(B)、Unlock(B)、Xlock(C)、Unlock(A)、Unlock(C)

  not obey the two-phase locking protocol

- 并行执行的所有事务均遵守两段锁协议，则对这些事务的所有并行调度策略都是可串行化的。

所有遵守两段锁协议的事务，其并行执行的结果一定是正确的

- 事务遵守两段锁协议是可串行化调度的充分条件，而不是必要条件
- 可串行化的调度中，不一定所有事务都必须符合两段锁协议。